

mightyZAP API Manual for Visual C++

(2010 / 2015 / 2017 version)

- Force Control version actuator (model name starts with 12Lf) users can use all the following parameters.
- The parameter marked **[F/C version only]** applies only to the Force Control version actuator (model name starts with 12Lf) and does not apply to the Position control version actuator.
- Position Control version actuator (model name starts with L12 or D12) user can use all parameters except **[F/C version only]** parameters.

• Initialization – COM Port Connection

`void OpenMightyZap(LPCTSTR portname, DWORD BaudRate)`

Function: Port connection for servo actuator control.

Parameter :

- `LPCTSTR`portname : Put Com Port number like a"COM3"
- `DWORD`BaudRate : Designate Baudrate as defined word as below

Defined Word	Baud rate
CBR_9600	9600
CBR_19200	19200
CBR_57600	57600
CBR_115200	115200

• Termination

`void CloseMightyZap()`

- Function: Close (terminate) port connection which controlled servo actuator.
- Parameter :N/A

• Restart

`void Restart(intbID)`

Function : Restart actuator system.

Shutdown due to overload, input voltage error, etc. can be restarted.

Parameter :

- `int` bID : Servo ID (1~253), BroadCAST ID(254), Stand-alone ID (0)
(herein after, explanation about servo ID to be omitted)

• Moving to Goal Position

`void GoalPosition(intbID, short position)`

Function:Move stroke position to the desired Goal position

Parameter :

- `int` bID :servo ID
- `short` position : Desired position (0~4095)

• Moving Speed Adjustment **[F/C Version Only]**

```
void GoalSpeed(int bID, short speed)
```

Function: Moving speed adjustment of servo actuator

Parameter :

- int bID : servo ID

- short speed : Set moving Speed (0~1023)

(0 : Max speed, 1~1023 Speed adjustment (1:low, 1023:high))

• Goal Current Adjustment **[F/C Version Only]**

```
void GoalCurrent(int bID, short current)
```

Function: Limit max current of servo actuator

Parameter :

- int bID : servo ID

- short current: Set Max current limit (0~1600)

※ Speed will be decreased when current value is decreased.

※ The value represents current[mA] and has +/-15% tolerance.

※ High Goal current setting may affect to the lifespan of servo actuator. Please refer to the data sheet for proper use.

• Read Present Position

```
void PresentPosition(int bID, short &pose)
```

Function: Read present stroke position

Parameter :

- int bID : servo ID

- short &pose : present stroke position to be saved in the "pose" as a reference variable for feedback. (Return '-1' value in case of Timeout)

• Acceleration Control **[F/C Version Only]**

```
void Acceleration(int bID, unsigned char accel)
```

Function: Control acceleration rate of servo actuator

Parameter :

- int bID : servo ID

- unsigned char accel: Acceleration rate value of actuator (1~255)

The higher the value, the faster the acceleration.

• Deceleration Control **[F/C Version Only]**

```
void Deceleration(int bID, unsigned char decel)
```

Function: Control deceleration rate of servo actuator

Parameter :

- int bID : servo ID / -unsigned char decel: Deceleration rate value of actuator (1~255)

The larger the value, the larger the deceleration rate and the slower the deceleration time.

※ If it is set too large, overshoot and minute vibration may occur when reaching the final position.

※ If overshoot occurs while changing the basic settings of the servo motor such as Speed and GoalCurrent, you can reduce the deceleration rate.

• Force On / Off

`void ForceEnable(int bID, bool enable)`

Function: Activate/Inactivate actuator Force (Motor ON/OFF)

Parameter :

- `int` bID : servo ID

- `bool` enable : 0 - Force Off / 1 - Force On

※ Communication is still alive even if actuator is under Force Off status.

※ Automatically Force ON when new goal position command is made.

What is the Force Off?

The actuator lineup having rated load 35N or more, have a characteristic to keep the position due to the mechanical design characteristics without motor power.

Therefore, if the servo actuator in the facility needs to maintain a certain position for considerable time, you can extend the life of the actuator by shutting off the motor power with the Force Off command. In this case, the communication is still maintained, and only the motor is powered off. When the position movement command is given again, the force is automatically turned on and the next command is executed.

• Shutdown On / Off

`void SetShutDownEnable(int bID, unsigned char flag)`

Function: Setting for Shutdown function On/Off for respective error

Parameter :

- `int` bID : servo ID

- `unsigned char` flag : Put setting value (See the mightyZAP user manual - Alarm shutdown)

• Read Shutdown status

`void GetShutDownEnable(int bID, unsigned char &flag)`

Function: Reading shutdown On/Off status for error

Parameter :

- `int` bID : servo ID

- `unsigned char` &flag : Setting condition to be saved in the "flag" as a reference variable.

(Return '-1' value in case of Timeout)

• LED Error Indication

`void SetErrorIndicatorEnable(int bID, unsigned char flag)`

Function: Setting for LED alarm indications On/Off for the respective errors.

Parameter :

- `int` bID : servo ID

- `unsigned char` flag : Put setting value

(See the mightyZAP user manual - Alarm LED)

• Read LED Error Indication Status

```
void GetErrorIndicatorEnable(int bID, unsigned char &flag)
```

Function: Read LED indications On/Off status for error

Parameter :

- int bID : servo ID
 - unsigned char &flag : setting condition to be saved in the "flag" as a reference variable.
- (Return '-1' value in case of Timeout)

• Read Error

```
void ReadError(int bID, unsigned char &error)
```

Function: Read current error status

Parameter :

- int bID : servo ID
 - unsigned char &error : error status to be saved in the "error".
- (Return '-1' value in case of Timeout)

• Direct Write to Address

```
void Write_Addr(int bID, unsigned char Addr, unsigned char Size, short Data)
```

Function: Writes data directly to the certain address of parameter(memory/parameter) in data map.

Parameter :

- int bID : Servo ID
 - unsigned char Addr: Parameter address to input data
 - unsigned char Size: Input data size
 - short Data: Input Data
- (See the mightyZAP user manual –Data Map)

• Direct Read from Address

```
void Read_Addr(int bID, unsigned char Addr, unsigned char Size, short & Data)
```

Function: READ data directly from the certain address of parameter(memory/parameter) in data map.

Parameter :

- int bID : Servo ID
- unsigned char Addr: Address to read
- unsigned char Size: Read Address byte Size
- short & Data: The variable to save the read data.

(Return '-1' value in case of Timeout)

(See the mightyZAP user manual –Data Map)

• Save data to same address in multiple actuators

```
void Sync_write_data(unsigned char Addr, unsigned char *data, unsigned char size)
```

Function : Save data to same address in multiple actuators

Parameter :

- unsigned char Addr : Parameter address to write data, See Factor#1 in the table below
- unsigned char *data : Input data matrix,

Enter factor values as a matrix after factor #2 in the table below

Length + ID1 + Data#1, +Data#2 +ID2 + Data#1 + data#2

- **unsigned char** Size : Input Data Size

- Servo ID 1 Goal Position : 1023(0x03FF), Servo ID 2 Goal Position : 2047(0x07FF)

HEADER	ID	Size	Command	Factor #1 addr	Factor #2 Length
0xFFFFFFFF	0xFE	0x0a	0x73	0x86	0x02

Factor #3 1> ID	Factor #4 1> Data #1	Factor #5 1>Data #2	Factor #6 2>ID	Factor #7 2> Data #1	Factor #8 2> Data #2	Checksum
0x01	0xff	0x03	0x02	0xFF	0x07	0xF1

(For more details, See the mightyZAP user manual –Symmetric Store)

• 1 byte transmission

void WriteByte(**unsigned char** Data)

Function : Send a single data to the actuator. Used when the user configures and sends the packet.

Parameter :

- **unsigned char** Data : 1byte Data to be transmitted to actuator

• Direct packet transmission

void WritePacket(**unsigned char** *buffer, **unsigned char** size)

Function : Use when the user directly constructs a packet and sends one completed packet to the servo actuator

Parameter :

- **unsigned char** buffer : Packet buffer to be sent to the actuator

- **unsigned char** size: Packet buffer size to be sent to the actuator

(See the mightyZAP user manual – Packet Description and Packet examples)